

AI Assistant ■

Setup, tools, diagrams, Desktop vs Web, troubleshooting, CLI details, and **workflows / use cases** for the BTF Viewer **AI** tab.

For day-to-day panel usage (templates, Apply/Skip, Analysis dialog buttons), see the [user guide → AI Assistant](#). Ask-order playbooks: [WORKFLOWS.md §7](#). End-to-end flows and symptom → tool tables: [Workflows and use cases](#).

Heading level: ■ section · ■ subsection · ■ topic · ■ detail

Contents ■

Section

How it works	Context, evidence, investigation plan
Workflows and use cases	End-to-end flows, symptom → tools, what-if / optimize
Endpoints and models	Ollama / cloud, context size, local models
GUI tools	Tool schema and Apply / Undo

Section

Diagrams	Mermaid and tables in replies
Desktop vs web	Parity matrix
Troubleshooting	CORS, auth, TLS, timeouts
Opening the web app from file://	Ollama CORS for disk pages
CLI regression gate	Headless analyze

How it works

The **AI** tab answers diagnostic questions using structured **Analysis Findings** and summary metrics—not the raw .btf event stream. That keeps the prompt compact (token-efficient) and analysis fast. The Trace Compare template sends those tables instead of Findings.

When a question needs granular per-task time-series, the model can call `query_raw_metric` to pull a scoped series on demand (still never the raw BTF file). `search_timeline` locates STI / tag / task timestamps like Find. `trigger_compare` returns Trace Compare CSV when two tabs are open. `detect_anomalies` ranks Findings as Critical / Warning / Info. `correlate_events` merges blocking / execution / migration / sync / priority events for one task. `compare_performance` returns structured A vs B deltas (two tabs). `regression_explain` narrates the primary A vs B change. `investigate` builds a root-cause chain plus hypotheses and suggested tools. `generate_report` returns typed engineering markdown (then `export_report` to save). `check_budget` compares WCET/response/deadline budgets; `optimize` returns qualitative mitigations; `what_if` runs a heuristic slice-replay simulator; `optimize_experiment` ranks automatic candidates; `analyze_traces`

ranks all open tabs; `bookmark_finding` pins semantic investigation marks; `investigation_replay` summarises a completed investigation. Query-only batches (`query_raw_metric`, `search_timeline`, `trigger_compare`, `investigate`, `detect_anomalies`, `correlate_events`, `compare_performance`, `generate_report`, `check_budget`, `optimize`, `regression_explain`, `investigation_replay`, `what_if`, `optimize_experiment`, `analyze_traces`) run immediately; mixed GUI batches wait for **Apply** unless **Auto-apply GUI actions** is on.

Important conclusions should cite evidence (jump:TIME, metric names) and state **confidence** (High / Medium / Low) plus evidence quality (Directly observed / Strong correlation / Possible explanation / Insufficient evidence). The system prompt asks the model not to invent numbers absent from findings, tool results, or Trace Compare tables.

Investigate / Root cause / What-if / Optimize / Diagnostic report show an **Investigation plan** checklist in the AI panel (steps advance as tools run; the final text reply marks remaining steps done). Analysis Findings may include anomaly rows (WCET Max>>Avg spikes, extreme migration bursts).

Natural-language timeline questions (e.g. “find STI wait around TaskA”) are answered via **search_timeline** — no separate search UI.

Recommended ask order ([WORKFLOWS.md §7](#)): triage overall findings → **Investigate / Root cause** when you want tool-driven drill-down → named metric templates → mitigations after the timeline agrees. Prefer the built-in templates; they already name the metrics and units. Concrete flows and use cases are below.

Workflows and use cases

Same flow on **Desktop** and **Web**. Panel chrome: [README](#) → [AI Assistant](#). Ladder and ask-order tables: [WORKFLOWS.md §7](#).

End-to-end flow

□ Load trace → Statistics (optional cursors + Limit to C1–Cn)

- Toolbar Analysis → Findings for that scope
- AI: Triage / Investigate / Root cause → Apply GUI cards, click jump:TIME
- Confirm on timeline + named Statistics sections
- What-if / Optimize (heuristic estimates) → only after the cause matches
↪ the timeline
- Diagnostic report / export_report → or CLI analyze for CI baselines

Do not ask for mitigations before the timeline agrees with the finding. Empty or mis-scoped Statistics produce confident nonsense.

Investigation workflow

Step	Template or tool	Why
1	Triage findings / detect_anomalies	Rank Critical / Warning / Info
2	Investigate / investigate	Root-cause chain, hypotheses, suggested tools
3	correlate_events + query_raw_metric	Merge blocking / execution / migrations / sync / PI for one task
4	set_cursors / zoom_to_range / highlight_task	Narrow the timeline (Apply unless auto-apply is on)

Step	Template or tool	Why
5	bookmark_finding	Pin root_cause / evidence / correlated / reference marks
6	investigation_replay / generate_report	Structured close-out; optional ex- port_report

Root cause walks deadline/WCET → preemption → blocking → mutex → inheritance → migration for the top finding. Use it when triage already named a suspect task.

What-if and optimize workflow

what_if and optimize_experiment are **heuristic slice-replay** tools: they reallocate measured execution slices, scale migrations / blocking, and adjust core-util balance. They are **not** a FreeRTOS kernel or deterministic scheduler. Every result carries a disclaimer.

Goal	What to run	Typical change phrases
One concrete idea	What-if → what_if	pin CS[28] to Core_0, raise priority of Low[266], reduce mutex contention 50%
Rank several ideas	Optimize → optimize_experiment (then optional optimize for qualitative notes)	Host picks pin-to-dominant / quiet core, contention –50%, priority up, migrations –50%

Goal	What to run	Typical change phrases
Soft advice only	optimize	Finding-text mitigations without scored experiments

Reading the payload: compare baseline vs simulated (migrations, blocking_ns, load_balance_score) and the deltas.cost ranking. Lower cost is better in the experiment list. Treat Medium confidence as “worth trying on the bench”; Low means the phrase was vague or slices were thin.

Use cases

Situation	Before you ask	Template / tools	Then verify
Unknown — first look	Full-trace or cursor-scoped Findings	Triage findings → Investigate	Named Statistics sections; jump: TIME
Migration thrash / ping-pong	Scope thrash window; Findings mention the task	Migration thrash → correlate_events → What-if pin / Optimize	Migrations Rate/Ping; Heatmap / Chord; Core Affinity
High blocking / mutex wait	Scope the stall; suspect task known	Highest latency → query_raw_metric blocking/sync → What-if contention / priority	Blocking Max; Mutex hold; Priority Inheritance
WCET / deadline pressure	Thresholds set in Display → Analysis	WCET / hot CPU or Deadline / budget → check_budget	Execution Max jump; Deadlines section

Situation	Before you ask	Template / tools	Then verify
Load imbalance across cores	Multi-core util in Findings	Core balance → <code>analyze_traces</code> (multi-tab) or What-if pin to quiet core	Load Balance Score; Concurrent Active; Core Time Breakdown
A vs B build regression	Two tabs open	Trace Compare → <code>compare_performance / regression_explain</code>	Trace Compare pages; same scope on both builds
Rank all open traces	≥2 loaded tabs	<code>analyze_traces</code>	Best tab vs Migrations / LB / missed ticks
Write-up for a review	Cause already confirmed	Diagnostic report → <code>generate_report</code> → <code>export_report</code>	Saved HTML/CSV; evidence times bookmarked
CI gate vs baseline	Desktop CLI	<code>analyze --fail-on-regression (optional --ai)</code>	Exit code + markdown narrative

Worked examples

Migration thrash → pin affinity

1. Place cursors on the thrash window; enable **Limit to C1-Cn**; re-open Analysis.
2. Run **Migration thrash** or **Investigate** until the hot task (e.g. CS[22]) and cores match the heatmap.
3. Ask **What-if**: *pin CS[22] to its dominant core* (or run **Optimize** for ranked candidates).

4. Read Δ migrations / Δ load_balance_score. If migrations drop but LB worsens sharply, try pin-to-quietest via `optimize_experiment` and compare ranks.
5. On firmware: set affinity / reduce bounce; re-capture a .btf and **Trace Compare** the before/after tabs.

Mutex contention → shorter critical section ■

1. **Highest latency** / `correlate_events` for the waiter; confirm hold episodes in Mutex / Priority Inheritance.
2. **What-if**: *reduce mutex contention 50% for TASK* (or another %).
3. Expect lower `blocking_ns` in the simulated payload — still an estimate. Confirm by shortening the hold in code and re-tracing.

Two builds → regression narrative ■

1. Open baseline and candidate as tabs; match cursor scope if needed.
2. **Trace Compare** template, or `compare_performance` then `regression_explain`.
3. Act only on High/Medium confidence deltas that Statistics on both tabs reproduce.
4. Optional: `optimize_experiment` on the candidate's hottest task to sketch mitigations (still heuristic).

Simulator limits ■

Does	Does not
Replay measured slices / migrations / blocking gaps for the Statistics scope	Run FreeRTOS scheduling, ISRs, or cache models
Score pin / priority / contention / migration experiments	Guarantee WCET or deadline after a firmware change

Does	Does not
Label every result as estimate / not measured	Replace timeline verification or a new capture

Phrase changes as **pin / affinity / priority / mutex / migration** so the simulator engages; vague text falls back to a qualitative estimate (simulator: none).

Endpoints and models

Any OpenAI-compatible endpoint works, including Ollama (<http://localhost:11434/v1>). Chat requests time out after 120s (**Stop** still cancels sooner). Give the endpoint at least an **8k** context window so a full Findings card plus a tool round still fits.

Local models: the shipped Ollama default is phi4-mini:3.8b (light triage). For reliable native function calling prefer qwen2.5:7b / qwen2.5:14b or llama3.1:8b (or larger). 3B-class models often skip tools and emit ``btftool fences instead.

Import presets: [examples/ai](#) ships [ollama.json](#), [gemini.json](#), [openai.json](#), [deepseek.json](#), [grok.json](#), and [presets.json](#). Imported values fill **Settings** → **AI**; review and confirm to save. Each preset keeps its own base URL, model, key, auth mode, and TLS flag.

Keys may also come from the environment as OPENAI_API_KEY, GEMINI_API_KEY, or OLLAMA_API_KEY (VITE_* prefixed on the web). A local Ollama endpoint needs no key.

GUI tools

The model may call several tools in one turn; they apply as a single batch. With **Auto-apply GUI actions** off (default in **Settings → AI**), each mutating batch shows **Apply / Skip** and **Undo**, plus **Apply GUI actions** under the log. Read-only `query_raw_metric` / `search_timeline` / `trigger_compare` / `investigate` / `detect_anomalies` / `correlate_events` / `compare_performance` / `generate_report` / `check_budget` / `optimize` / `regression_explain` / `investigation_replay` / `what_if` / `optimize_experiment` / `analyze_traces` batches run immediately (no Apply card).

Tool	Parameters / targets	Effect
<code>set_cursors</code>	<code>timestamps</code> (1-8 trace times)	Place cursors (enables Limit to C1-Cn when two or more)
<code>zoom_to_range</code>	<code>start_time</code> , <code>end_time</code>	Focus the timeline between two times
<code>highlight_tasks</code>	<code>task_name_or_id</code> (display name, numeric id, or merge key)	Lock-highlight a task row. Unknown names are ignored so the timeline is not dimmed. Empty string clears.
<code>set_view_mode</code>	<code>mode</code> (task / core); optional orientation	Switch Task or Core view; horizontal or vertical
<code>open_corridor_inspector</code>	<code>core_from</code> / <code>core_to</code> (Core_0, 0, c0, Core 0)	Open Migration Inspector; aliases resolve the same way

Tool	Parameters / targets	Effect
add_annotation	time, note (≤ 240 chars)	Pin an orange timeline note at a timestamp (stays on the current right-panel tab)
query_raw_metrics	task, metric (priority_inheritance, execution, migrations, blocking, sync, findings)	Read-only: return the per-task series for the current Statistics scope (up to 40 rows)
export_report	optional format (html / csv)	Download HTML or CSV bundling Analysis Findings, mermaid diagrams from the chat, annotations, and GUI state (cursors / highlight / view).
clear_marks	optional what (annotations / cursors / bookmarks / all / everything)	Clear AI clutter. all (default) drops annotations + cursors; everything also clears bookmarks

Tool	Parameters / targets	Effect
reset_view	(none)	Fit the timeline to the full span and clear the task highlight (marks stay)
search_timeline	query; optional mode (contains / exact / regex / sti / tags / intervals / lifecycle / pointers / migrations)	Find-panel search; returns matching timestamps (up to 40)
trigger_compare	optional tab_a / tab_b (0-based tab index or filename)	Read-only Trace Compare CSV + open the compare dialog (needs two loaded tabs)
investigate	optional finding_id, depth (1-5)	Read-only: investigation graph with root-cause chain, hypotheses, ranked anomalies, suggested tools
detect_anomalies	optional limit (1-40)	Read-only: rank Analysis Findings as Critical / Warning / Info
correlate_events	task; optional around_time, window	Read-only: merge blocking / execution / migration / sync / priority / Find hits into one timeline

Tool	Parameters / targets	Effect
compare_performance	optional tab_a / tab_b	Read-only: structured A vs B metric deltas + confidence (two tabs)
generate_report	optional report_type, finding_id	Read-only: typed engineering markdown (executive / performance / root_cause / regression / optimization / bug / ci); call export_report to save
check_budget	optional budgets, tasks	Read-only: compare per-task WCET/response/deadline metrics against budgets (host builds rows from findings when tasks omitted)
optimize	optional limit (default 5)	Read-only: evidence-backed mitigation ideas (estimate disclaimer)
regression_experiment	optional tab_a / tab_b	Read-only: compare two tabs then narrate the primary regression

Tool	Parameters / targets	Effect
bookmark_finding	time, kind (root_cause / evidence / correlated / reference); optional note	GUI: pin a semantic investigation annotation (Apply required)
investigation_replay	optional finding_id, conclusion, tools_run, evidence_times	Read-only: structured investigation replay card
what_if	change; optional task	Read-only: heuristic slice-replay what-if (migrations / blocking / load balance; not FreeRTOS kernel)
optimize_experiment	optional task, limit (1-12, default 5)	Read-only: run ranked automatic pin/priority/contention/migration experiments
analyze_trace	(none)	Read-only: rank all loaded tabs by scheduling behavior

Models without native tool calling can emit a fenced ```btf tool JSON block (same cards). Prefer a tool-capable model (see [Endpoints and models](#), or gpt-4o / Gemini) if native calls stay silent.

After **Apply**, **Undo last actions** restores zoom / view / highlight / inspector / marks; **Ctrl/Cmd+Z** also reverts cursors and marks.

Diagrams ■

Replies may include ``mermaid **sequence** diagrams (mutex take/give, block/resume, priority boost / L/M/H) and graph LR / **flowchart** core-migration graphs (counts on edges). Pipe **Markdown tables** (and a sanitized HTML <table> copied from Findings) render as HTML tables in the reply pane.

- Click a **task** node to lock-highlight that timeline row (Low[266] (Core 0) resolves to Low[266])).
 - Click a **core** node (Core_0, C0, C1) to switch to Core View and scroll to that core.
 - Mutex hex and other unresolved labels do nothing (the timeline stays undimmed).
 - Click empty figure area to open a larger zoom window (scroll to zoom 0.5-6×; **Esc** or **Close**). Trackpad pinch is treated as scroll.
 - The link row under the figure has the same targets.
 - **Save As...** HTML keeps inline SVG with clickable nodes (chat zoom wrappers are omitted).
-

Desktop vs web ■

Area	Desktop	Web
Native tools + ``btftool	Same schema and cards	Same
add_annotation / query_raw_metric / export_report	Marks + scoped series + save dialog	Same (browser download)

Area	Desktop	Web
clear_marks / reset_view / search_timeline / trigger_compare / investigate / detect_anomalies / correlate_events / compare_performance / generate_report / check_budget / optimize / regression_explain / investiga- tion_replay / what_if / optimize_experiment / analyze_traces / bookmark_finding	Same	Same (compare overlay; search uses Find; investigation tools are read-only except book- mark_finding)
highlight_task / corridor cores	Same resolve rules	Same
In-chat mermaid figure	Data-URI image + node hit-test	Inline SVG node clicks
In-chat Markdown / HTML tables	Same rendered table	Same
Zoom window + link row	Scroll to zoom + link row	Same
Authentication	Settings → AI → None / API key / Sign in; panel chip + 401 CTAs until a successful turn	Same (VITE_* env keys)

Area	Desktop	Web
Self-signed TLS	Allow self-signed TLS per preset skips HTTPS certificate checks	Persist the same flag + tip; browsers still verify — trust the cert, use http://, or use Desktop
Model picker	Editable combo; refresh fills and opens the dropdown	Same
Fonts	pt	px
Endpoint from file://	N/A	CORS — prefer Vite proxy, or Opening the web app from file://

Troubleshooting

Symptom	Cause	Try
Web: Failed to fetch / CORS	Browser blocked a cross-origin call (file:// sends Origin: null)	Prefer npm run dev / make preview (both proxy Ollama), or see Opening the web app from file://

Symptom	Cause	Try
401 / 403	Missing or rejected key / origin	Settings → AI → Sign in or API key (OPENAI_API_KEY / GEMINI_API_KEY / OLLAMA_API_KEY; local Ollama needs none)
CERTIFICATE_VERIFY_FAILED / self-signed TLS	Private CA or self-signed HTTPS gateway	Desktop: Settings → AI → Allow self-signed TLS . Web: trust the cert in the OS/browser, use http:// on a private LAN, or use the Desktop app

Symptom	Cause	Try
Chat probe timed out / The read operation timed out	GET /models lists ids only; inference is slow or hung	Test connection POSTs /chat/completions (non-streaming, 120s). Warm the model (ollama run MODEL) and retry. Debug with the curl probe below; if curl hangs too, the gateway's chat upstream is stuck. Try "stream": true if non-stream never returns. Lower context length on a VRAM-tight local host.
Model not found	Typed id is not served	Refresh the Model list (or Test connection) and pick a served id from the dropdown, or ollama pull it
Gemini HTTP 400 thought_signature	Gemini 3 requires a thought blob on tool follow-ups	Retry the question — the viewer echoes Gemini thought signatures

Symptom	Cause	Try
Raw ``btftool JSON instead of native tool calls	Model lacks (or skips) function calling	Same cards either way — Apply or enable Auto-apply GUI actions . Switch to qwen2.5:7b / llama3.1:8b / gpt-4o / Gemini for native calls
Ask times out (over 120s) or stays on Waiting...	Cold start, CPU offload, or VRAM spill	Stop , warm with ollama run MODEL, retry. Use Clear between long threads. Smaller model or shorter Statistics scope if the Findings card is huge
Later turns ignore earlier facts	Chat history exceeded the context window	Clear on the AI bar, or Analysis → Query with AI... for a fresh scoped prompt

Test connection

Same body the viewer sends for **Test connection** (replace BASE, MODEL, and KEY):

```
curl -vk --max-time 180 \  
  -H "Authorization: Bearer KEY" \  
  -H "Content-Type: application/json" \  
  -d {}
```

```
-d  
↪ '{"model":"MODEL","stream":false,"messages":[{"role":"user","content":"Reply  
↪ with exactly: OK"}],"max_tokens":8}' \  
BASE/chat/completions
```

Opening the web app from file://

A page opened straight from disk sends `Origin: null`, which Ollama rejects with 403 — the browser then reports only `Failed to fetch`. Serving the app over `http` avoids this entirely (`npm run dev / make preview proxy Ollama for you`), and the Desktop app is not affected at all.

To keep using `file://`, allow every origin on the Ollama side:

Server started from a terminal

```
OLLAMA_ORIGINS="*" ollama serve
```

macOS menu-bar app (Ollama.app) – a shell variable does not reach it

```
launchctl setenv OLLAMA_ORIGINS "*" # then quit Ollama and reopen it
```

Verify the change took effect; expect 200 and an `Access-Control-Allow-Origin` header:

```
curl -s -D - -o /dev/null -H "Origin: null" http://localhost:11434/v1/models  
↪ \  
| grep -iE "^HTTP|access-control-allow-origin"
```

If a `file://` page is still refused, list the null origin explicitly with `OLLAMA_ORIGINS="*,null"`. Note that `*` lets **any** page you visit reach your local models; undo it with `launchctl unsetenv OLLAMA_ORIGINS` when done.

CLI regression gate

Desktop headless CI can compare a candidate trace to a baseline and optionally ask the configured AI for a short narrative:

```
python builds/btf_viewer.py analyze candidate.btf --baseline baseline.btf
↳ --fail-on-regression
python builds/btf_viewer.py analyze candidate.btf --save-baseline
↳ /tmp/base.json
python builds/btf_viewer.py analyze candidate.btf --baseline /tmp/base.json
↳ --fail-on-regression --ai
```

See also [Export → Headless CLI](#) in the user guide.